

Практическая работа 6

Сегментация изображений

Сегментация изображения — это процесс разделения цифрового изображения на несколько отдельных областей, содержащих каждый пиксель (наборы пикселей, также известные как суперпиксели) с аналогичными атрибутами.

Цель сегментации изображения состоит в том, чтобы изменить представление изображения во что-то более значимое и более простое для анализа.

Сегментация изображения обычно используется для поиска объектов и границ (линий, кривых и т. д.) в изображениях. Точнее говоря, сегментация изображения - это процесс назначения метки каждому пикселю в изображении, чтобы пиксели с одинаковой меткой имели определенные характеристики.

Сегментация изображения включает в себя преобразование изображения в набор областей пикселей, которые представлены маской или помеченным изображением. Разделив изображение на сегменты, вы можете обрабатывать только важные сегменты изображения вместо обработки всего изображения.

Обычный метод заключается в поиске резких разрывов в значениях пикселей, которые обычно указывают края, которые определяют область.

Другим распространенным подходом является обнаружение сходства в областях изображения. Некоторые методы, которые следуют этому подходу, включают в себя рост регионов, кластеризацию и пороговое определение.

Пороговая сегментация

Процесс определения *порога* включает сравнение каждого значения пикселя изображения (интенсивности пикселя) с заданным порогом. Это делит все пиксели входного изображения на 2 группы:

1. Пиксели со значением яркости ниже порогового.
2. Пиксели, имеющие значение интенсивности больше порогового.

Этим двум группам теперь присваиваются разные значения в зависимости от различных типов сегментации.

OpenCV поддерживает 5 различных схем пороговой обработки для изображений в градациях серого (8-битные) с помощью функции:

cv.threshold (InputArray src, OutputArray dst, thresh, maxval, type int)

Параметры:

- Первый параметр, src типа InputArray, выходной массив, одиночный канал, 8 или 32-битные объекты Mat;
- Второй параметр, dst типа OutputArray, выходной массив, имеет тот же размер и тип, что и src;
- Третий параметр, двойной тип thresh, конкретное значение порога;
- Четвертый параметр, maxval типа double, когда пятый параметр принимает пороговый тип THRESH_BINARY или THRESH_BINARY_INV;
- Пятый параметр type типа int - это метод порогового изображения, значения которого следующие:
 1. THRESH_BINARY: пороговое значение с двумя значениями ...
 2. THRESH_TOZERO_INV: обратный двоичный порог 1
 3. THRESH_TRUNC: пороговое значение усечения
 4. THRESH_TOZERO: ниже порогового значения установлено значение 0
 5. THRESH_TOZERO_INV: превышение порога установлено в 0

Типы пороговых значений

- **Двоичный порог (int type = 0)**

Из двух групп, полученных ранее, группе, имеющей элементы с интенсивностью пикселей, превышающей установленный порог, присваивается «Max_Value» или, в случае градаций серого, значение 255 (белый).

У членов оставшейся группы интенсивность пикселей установлена на 0 (черный).

Если значение интенсивности пикселя в (x, y) в исходном изображении больше порогового значения, значение в конечном изображении устанавливается на «maxVal».

- **Инвертированный двоичный порог (int type = 1)**

Инвертированный двоичный порог такой же, как Двоичный порог. Единственное существенное отличие состоит в том, что при установке порога InvBinary группе, имеющей интенсивность пикселей больше установленного порога, присваивается «0», тогда как остальным пикселям, имеющим интенсивности, меньшие порогового значения, устанавливается значение «maxVal».

Если значение интенсивности пикселя в (x, y) в исходном изображении больше порогового значения, значение в конечном изображении устанавливается на «0», иначе оно устанавливается на «maxVal».

- **Усечение порога (int type = 2)**

Группа, имеющая интенсивности пикселей, превышающие установленный порог, усекается до установленного порога или, другими словами, значения пикселей устанавливаются такими же, как установленный порог.

Все остальные значения остаются прежними.

Если значение интенсивности пикселя в (x, y) в исходном изображении больше порогового значения, значение в конечном изображении устанавливается равным «порогу», в противном случае оно не изменяется.

- **Порог до нуля (int type = 3)**

Очень простой метод определения порога, при котором мы устанавливаем интенсивность пикселя на «0» для всех пикселей группы, имеющих значение интенсивности пикселя, меньшее порогового значения.

Если значение интенсивности пикселей в (x, y) в исходном изображении больше порогового значения, значение в (x, y) в конечном изображении не изменяется. Для всех остальных пикселей установлено значение «0».

- **Порог до нуля, инвертированный (int type = 4)**

Как и в предыдущем методе, здесь мы устанавливаем интенсивность пикселей на «0» для всех пикселей группы, имеющих значение интенсивности пикселей, превышающее пороговое значение.

Если значение интенсивности пикселя в (x, y) в исходном изображении больше порогового значения, значение в (x, y) в конечном изображении устанавливается на «0». Все остальные значения пикселей не меняются.

После обработки порогом изображения выявляются контуры в изображении с помощью функции

```
Cv2.findContours( Image, mode, method [, contours[, hierarchy[, offset]])
```

Image — должным образом подготовленная для анализа картинка. Это должно быть 8-битное изображение. Поиск контуров использует для работы монохромное изображение, так что все пиксели картинки с ненулевым цветом будут интерпретироваться как 1, а все нулевые останутся нулями.

mode — один из четырех режимов группировки найденных контуров:

- **CV_RETR_LIST** — выдаёт все контуры без группировки;
- **CV_RETR_EXTERNAL** — выдаёт только крайние внешние контуры. Например, если в кадре будет пончик, то функция вернет его внешнюю границу без дырки.
- **CV_RETR_CCOMP** — группирует контуры в двухуровневую иерархию. На верхнем уровне — внешние контуры объекта. На втором уровне — контуры отверстий, если таковые имеются. Все остальные контуры попадают на верхний уровень.
- **CV_RETR_TREE** — группирует контуры в многоуровневую иерархию.

method — один из трёх методов упаковки контуров:

- **CV_CHAIN_APPROX_NONE** — упаковка отсутствует и все контуры хранятся в виде отрезков, состоящих из двух пикселей.
- **CV_CHAIN_APPROX_SIMPLE** — склеивает все горизонтальные, вертикальные и диагональные контуры.
- **CV_CHAIN_APPROX_TC89_L1, CV_CHAIN_APPROX_TC89_KCOS** — применяет к контурам метод упаковки (аппроксимации) Teh-Chin.

contours — список всех найденных контуров, представленных в виде векторов;

hierarchy — информация о топологии контуров. Каждый элемент иерархии представляет собой сборку из четырех индексов, которая соответствует контуру[i]:

- **иерархия[i][0]** — индекс следующего контура на текущем слое;
- **иерархия[i][1]** — индекс предыдущего контура на текущем слое;
- **иерархия[i][2]** — индекс первого контура на вложенном слое;
- **иерархия[i][3]** — индекс родительского контура.

offset — величина смещения точек контура.

Для улучшения выявленных контуров используются морфологические операции:

Erosion
Dilation
Opening
Closing
Morphological Gradient
Top hat
Black hat

См. <https://bestprogrammer.ru/programmirovaniye-i-razrabotka/python-opencv-morfologicheskie-operatsii>

Для всех вышеперечисленных методов двумя важными требованиями являются бинарное изображение и элемент структурирования ядра, который используется для скольжения по изображению.

Алгоритм сегментации на основе кластеризации К-средних

Шаги в алгоритме К-средних:

1. Выберите количество кластеров K .
2. Выберите случайным образом K точек, центроиды (не обязательно из вашего набора данных).
3. Назначьте каждую точку данных ближайшему центроиду $\rightarrow$, который формирует K кластеров.
4. Вычислите и разместите новый центроид каждого кластера.
5. Переназначьте каждую точку данных на новый ближайший центроид. Если любая переназначение. состоялся, перейдите к шагу 4, в противном случае модель готова.

OpenCV обеспечивает `cv2.kmeans ( образцы, nclusters (K), критерии, попытки, флаги )` функция для цветовой кластеризации.

1. **образцы:** Это должно быть `np.float32` Тип данных, и каждая функция должна быть помещена в один столбец.
2. **кластеры (K):** Количество кластеров, необходимых в конце
3. **критерии:** Это критерий завершения итерации. Когда этот критерий удовлетворен, итерация алгоритма прекращается. На самом деле, это должен быть набор из 3 параметров. Это `(type, max_iter, epsilon)`:

Тип (type) критерия прекращения:

- `cv.TERM_CRITERIA_EPS`- остановить итерацию алгоритма, если указана точность, *эпсилон*, достигнут.
- `cv.TERM_CRITERIA_MAX_ITER`- остановить алгоритм после указанного количества итераций, *max_iter*,
- `cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER`- остановить итерацию при выполнении любого из указанных выше условий.

4. **попытки:** Флаг, указывающий, сколько раз алгоритм выполняется с использованием разных начальных меток. Алгоритм возвращает метки,

которые дают лучшую компактность. Эта компактность возвращается как результат.

5. флаги: Этот флаг используется для указания того, как берутся начальные центры. Обычно для этого используются два флага:

cv.KMEANS_PP_CENTERS а также **cv.KMEANS_RANDOM_CENTERS**,

Задание:

1) Довести программу, представленную ниже, до рабочего состояния.

Провести эксперимент по обработке двух разного типа фотографий, одной с людьми, другой – с искусственными объектами, для двух разных значений порога.

```
import cv2
import numpy as np

# Загрузите изображение
img = cv2.imread('image.jpg')

# Преобразуйте изображение в оттенки серого
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Примените пороговое значение для сегментации (можно настроить параметры)
_, thresh = cv2.threshold(gray_img, 127, 255, cv2.THRESH_BINARY)

# Примените морфологические операции для улучшения сегментации
kernel = np.ones((5,5), np.uint8)
morph_img = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)

# Выполните сегментацию путем нахождения контуров объектов на изображении
contours, _ = cv2.findContours(morph_img, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# Нарисуйте контуры на исходном изображении
segmented_img = img.copy()
cv2.drawContours(segmented_img, contours, -1, (0, 255, 0), 2)

# Отобразите результат
cv2.imshow('Segmented Image', segmented_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

В этом примере используются пороговое значение для сегментации, морфологические операции для улучшения сегментации и функция `findContours` для нахождения контуров объектов на изображении. После этого выводятся контуры на исходном изображении.

2) Довести программу, представленную ниже, до рабочего состояния.

Провести эксперимент по обработке двух разного типа фотографий, одной с людьми, другой – с искусственными объектами, для трех разных значений K.

```
import cv2
import numpy as np
```

```

# Загрузите изображение
img = cv2.imread('image.jpg')

# Преобразуйте изображение в массив NumPy и измените его форму
pixel_vals = img.reshape((-1,3))
pixel_vals = np.float32(pixel_vals)

# Установите параметры для кластеризации k-means
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.85)
k = 3

# Выполните кластеризацию k-means
_, labels, centers = cv2.kmeans(pixel_vals, k, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)

# Преобразуйте метки в форму, которую можно использовать для создания маски
сегментации
labels = labels.reshape((img.shape[0], img.shape[1]))
mask = np.zeros_like(labels)

# Нарисуйте маску сегментации
for i in range(k):
    mask[labels == i] = centers[i]

# Отобразите результат
cv2.imshow('Segmented Image', mask)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

В этом примере используется алгоритм кластеризации k-means для сегментации. Сначала изображение преобразуется в массив NumPy и устанавливаются параметры для кластеризации. Затем выполняется кластеризация k-means и преобразуются метки в форму, которую можно использовать для создания маски сегментации. Наконец, рисуется маска сегментации.

Выводы:

Сравнение алгоритмов.

Объяснить различия.